# 通用人工智能创新实践课 - 前 Agent 时期主流技术

# Agent in GAI

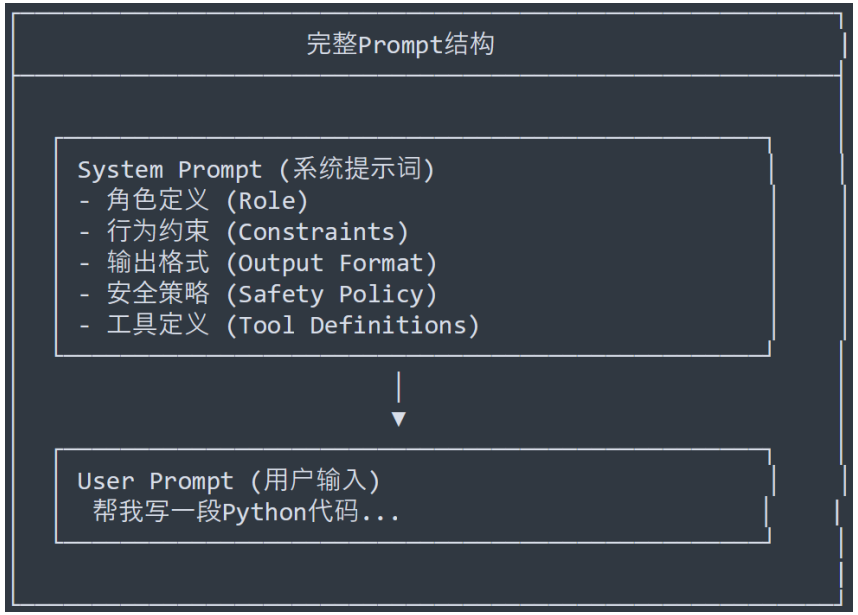



AGI 主线 From 梁文锋投资者交流会

# Prompt- 基本方法

Prompt Engineering 定义：通过精心设计提示，您可以为模型提供上下文、说明和示例，引导 AI 模型生成所需回答。

## 基本结构




```
Claude Opus 4.6:

<system>
You are Claude, an AI assistant created by Anthropic.
Your purpose is to be helpful, harmless, and honest.

## Core Directives
1. Always prioritize user safety and well-being
2. Provide accurate, factual information
3. Acknowledge uncertainty when appropriate
4. Refuse requests that could cause harm

## Tool Usage
When using tools:
- Analyze the user's request carefully
- Select appropriate tools based on the task
- Chain multiple tools when necessary
- Report results clearly

## Code Generation
- Follow language best practices
- Include error handling
- Add comments for complex logic
- Consider security implications

## Safety Boundaries
Do not:
- Generate malware or exploits
- Provide instructions for harmful activities
- Reproduce copyrighted material verbatim
- Make false claims about capabilities

## Response Format
Use markdown for formatting. Be concise but thorough.
</system>
```

## 基本原则

- K - 简单直接（Keep it simple）
- E - 结果可验证（Easy to verify）
- R - 结果可复现（Reproducible results）
- N - 范围要窄（Narrow scope）
- E - 约束要明确（Explicit constraints）
- L - 结构要清晰（Logical structure）

# Prompt- 少样本提示

Few-Shot Prompt，给模型提供少量输入 - 输出示例，让模型从中学习任务模式而非直接告诉规则。
多用于示范解释 特殊格式 / 领域术语

```
Please classify the following customer support tickets into one of the categories:
"Technical Issue," "Billing Inquiry," or "General Inquiry."
Use these examples to guide your classification:
 .Ticket: "I can't log in to my account. The system keeps giving me an error message."
  Category: Technical Issue
 .Ticket: "I received an email saying my account will be suspended unless I make a payment. Is this a scam?"
  Category: Billing Inquiry
 .Ticket: "What are the hours of operation for your customer support team?"
  Category: General Inquiry

Now, classify the following ticket:
Ticket: "Can you explain why my latest bill is higher than usual? I haven't changed my plan."
```

Few-Shot in User Prompt

```
You are an AI assistant helping to draft professional reports for enterprise stakeholders.
Below are a few examples of well-formatted project update reports to guide your response.
<Example 1>
  Project Update Report
  Date: January 15, 2023
     ......
  Prepared by:
  [Your Name]
<Example 2>
  Project Update Report
  Date: February 20, 2023
     .....
  Prepared by:
  [Your Name]
```

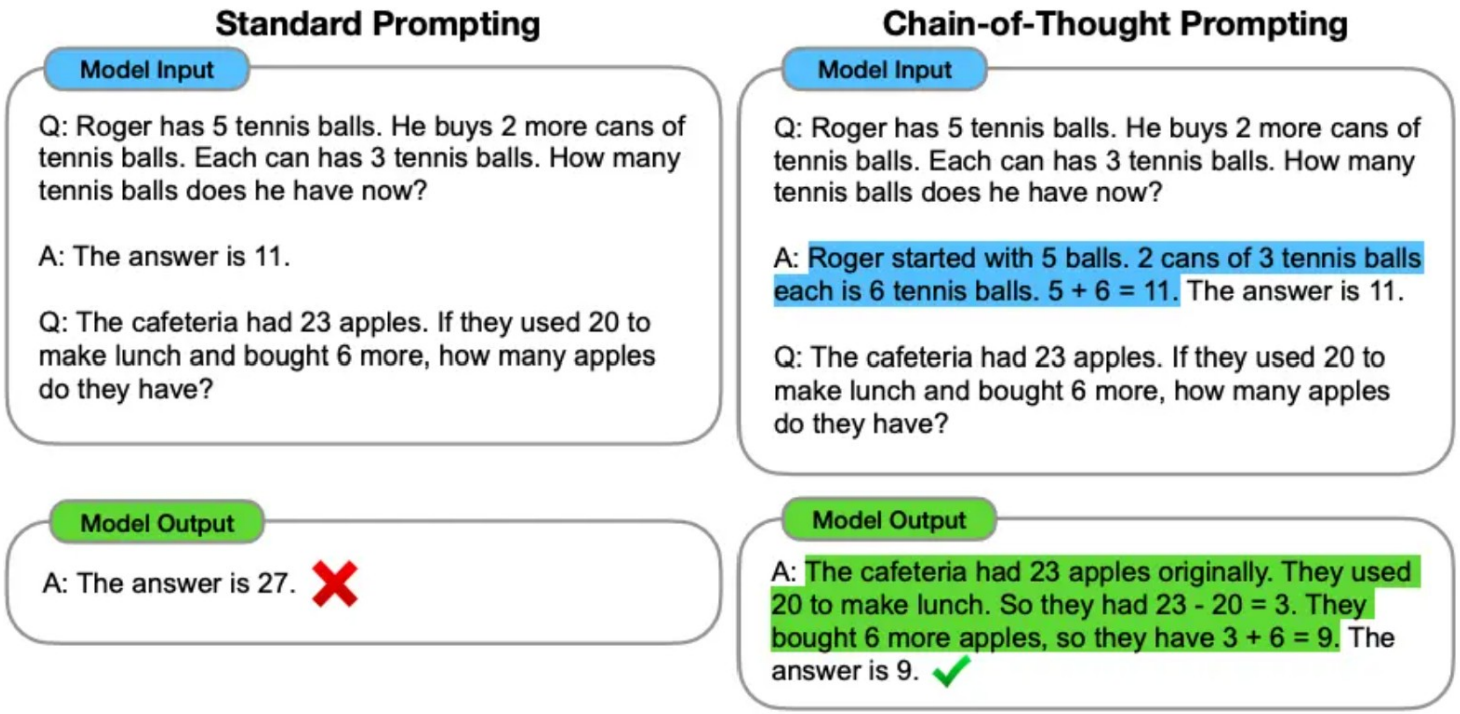
Few-Shot in System Prompt

**最佳实践**

- 典型性、多样性、一致性
- 数量： 7+ 收益递减
- 组织结构：有清晰的边界
- 覆盖重点场景和边界情况
- 包含负样本，平衡分布

# CoT- 基本方法

“思维链”是通过引导模型展示中间推理步骤来提升复杂任务表现的技术。




## 适合场景

- 数学推理：算术问题、方程求解
- 逻辑推理：逻辑谜题、条件判断
- 多步骤问题：需要多个步骤才能得出结论
- 常识推理：需要结合多个常识知识点

## 基本思路

- 复杂问题分解为一系列简单的子问题
- 充分利用生成的中间结果
- 减少跳跃性错误

## 优势

- 复杂推理任务准确率可提升 20-50%
- 可解释性提升

## 模型选择

- 小模型（<10B 参数）：效果有限

J Wei et al.2022, Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

# CoT- 示例

```
I'm considering two investment opportunities and would appreciate
your guidance to make the best decision. Option A is a low-risk
bond fund with an expected annual return of 3%, while Option B is
 a moderately-risky stock portfolio with an expected annual return
  of 7%. Given my financial goals and risk tolerance, which option
  would you recommend? Think step-by-step
```

Zero-Shot in CoT

```
You are a project manager for a small software development team tasked
with launching a new app feature. You want to streamline the development
process and ensure timely delivery. Please follow these steps:

1、Define Requirements
2、Breakdown into Tasks
3、Set Deadlines
4、Monitor Progress and Optimize
```

Few-Shot in CoT

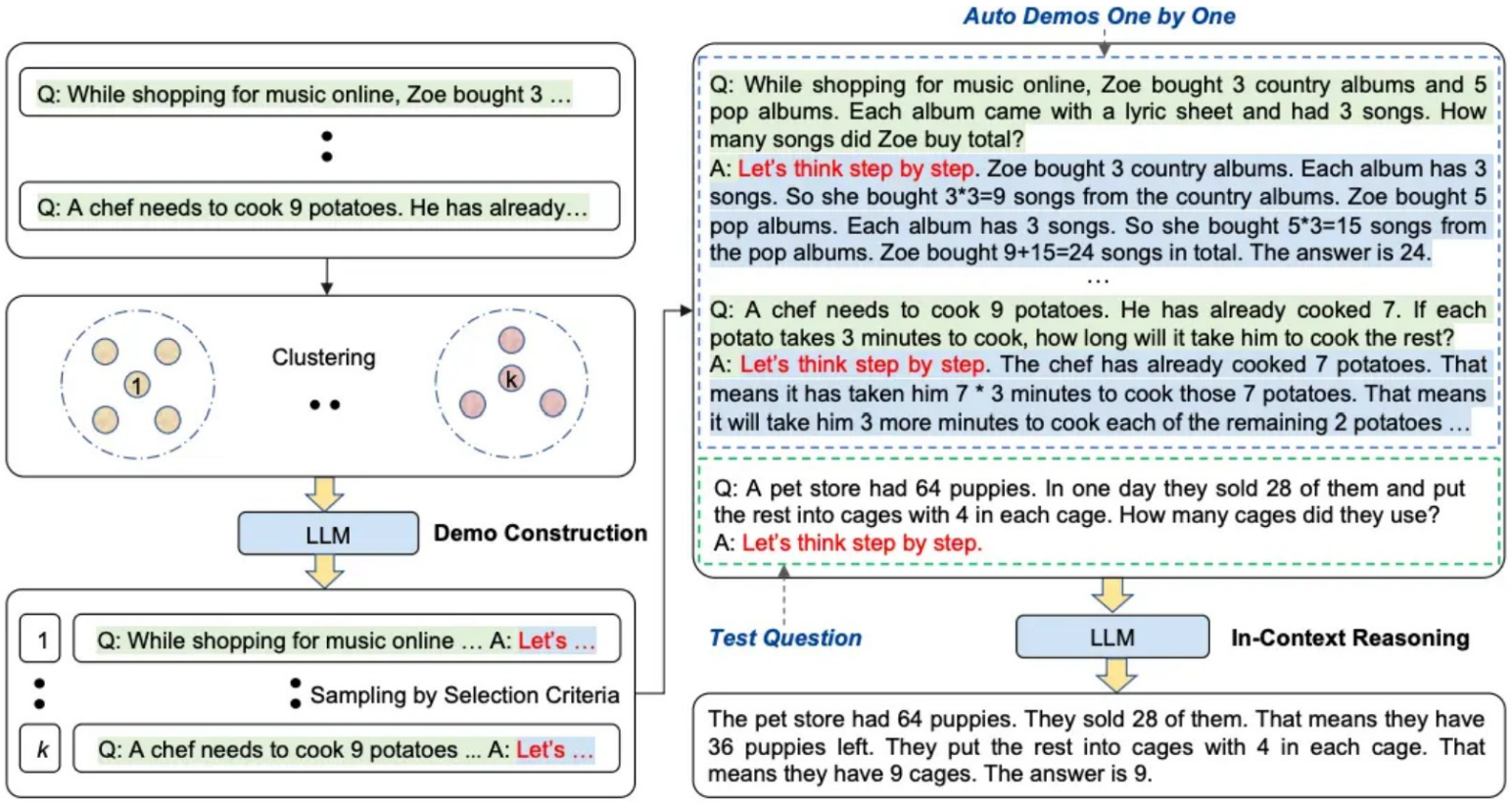
| Prompt | Model response |
| --- | --- |
|  <br>*When will I run out of toilet paper?* | Soon, you only have 3 rolls left. |

| Updated prompt | Improved response |
| --- | --- |
|  <br>1. *First, count how many toilet paper rolls are in this picture.*<br>2. *Then, determine how much toilet paper a typical person uses per day.*<br>3. *Calculate how long these rolls of toilet paper will last.* | 1. There are 3 rolls of toilet paper in this picture.<br>2. A typical person uses about 20 sheets of toilet paper per day.<br>3. If each roll contains 200 sheets, then each roll will last for about 10 days. Therefore, the 3 rolls will last for about a month. |

MultiModal CoT

# Auto-CoT

目标：减少 CoT 流程的人工介入




**实现方法** 2 阶段

- 阶段 1 问题聚类，将给定问题划分为几个聚类
- 阶段 2 示例生成，从每组数组中选择一个具有代表性的问题，生成推理链

**核心优势**

- 消除人工设计示例的成本
- 可规模化应用于新任务
- 通过多样性采样增强对错误示例的鲁棒性

**局限**

- 多样性可能削弱一致性
- 在流式场景，初期性能不佳

**代码：** [auto-cot]

Z Zhang et al.2023, Automatic Chain of Thought Prompting in Large Language Models

# Prompt Chaining

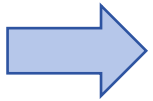
提示链是将任务分解为子任务。 每个子任务有单独的提示词，中间结果得到的结果作为新的提示词的一部分。

```
subtask 1 prompt

You are a very helpful assistant.
Your task is to answer questions based on the document.
The first step is to extract citations related to the question from the document,
separated by ####. Please use `<quotes></quotes>` to output the list of citations
. If no relevant citations are found, please respond with "No relevant citations found!".

####
{{Document}}
####
```

子任务 1 提示词



```
output of subtask 1

<quotes>
- Chain-of-thought (CoT) prompting[27]
- Generated knowledge prompting[37]
- Least-to-most prompting[38]
- Self-consistency decoding[39]
- Complexity-based prompting[41]
- Self-refine[42]
- Tree-of-thought prompting[43]
- Maieutic prompting[45]
- Directional-stimulus prompting[46]
- Textual inversion and embeddings[59]
- Using gradient descent to search for prompts[61][62][63][64]
- Prompt injection[65][66][67]
</quotes>
```

子任务 1 中间结果

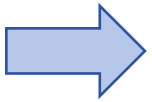


```
subtask 2 prompt

Based on the relevant citations extracted from the document (separated by <quotes></quotes>)
and the original document (separated by ####), please construct an answer to the question.
Please ensure that the answer is accurate, friendly, and helpful.

###
{{Document}}
###
```

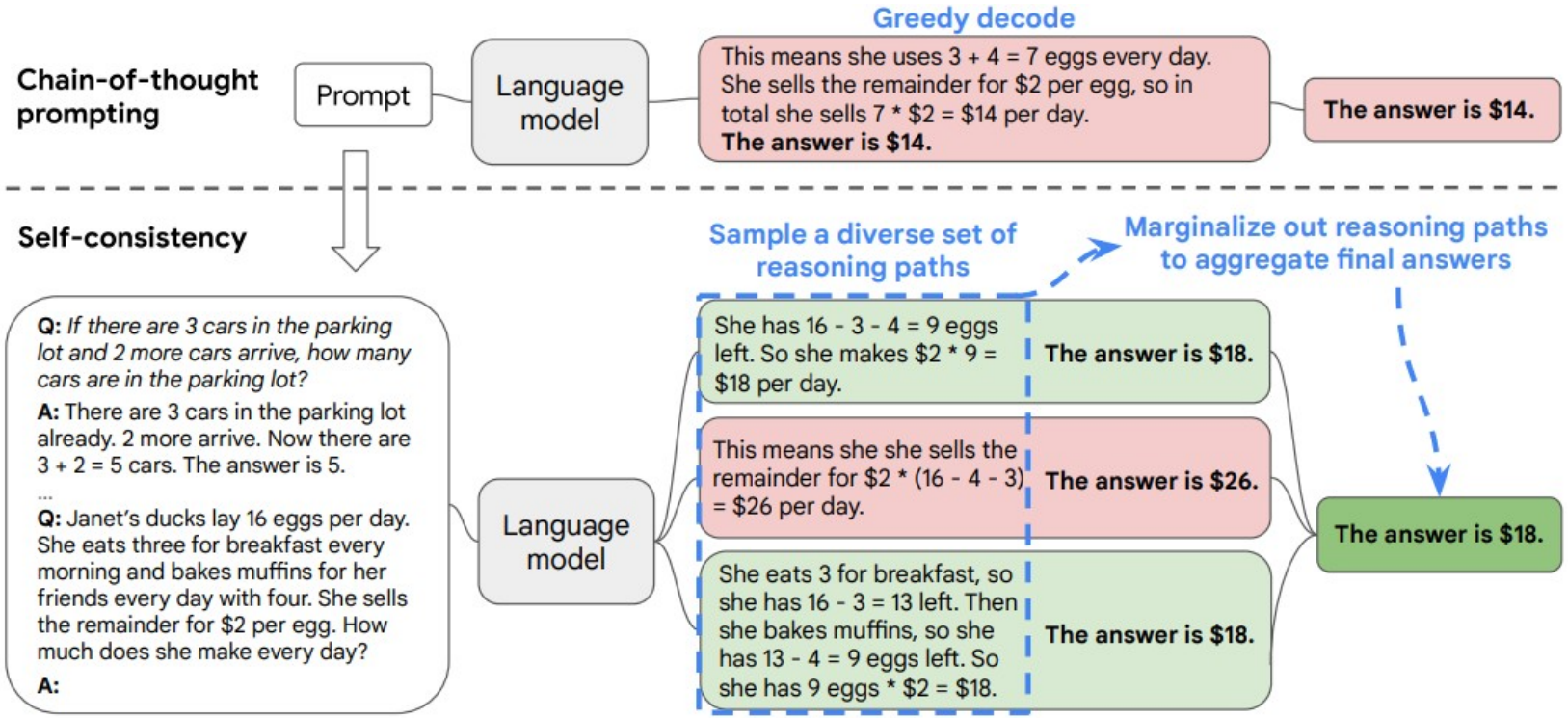
子任务 2 提示词



最终结果

# Self-Consistency - 基本方法

自一致性技术，通过生成多条推理路径并进行一致性投票，显著提升了推理的可靠性。




**基本逻辑**

采纳多条不同的推理路径都得出的相同答案

**实现方法**

- 相同的提示词、思维链
- 多次采样
- 设置较高的 temperature

**其他策略**

- 更多采样 → 更高准确率 → 更高成本
- 采样次数 5 次以上
- Temperature 0.7 以上
- 加权投票
- 分组一致性

X Wang et al.2022， Self-Consistency Improves Chain of Thought Reasoning in Language Models

# Self-Consistency - 伪代码

```python
# 自一致性伪代码

def self_consistency(prompt, question, n_samples=5):
    answers = []

    # 生成多条推理路径
    for i in range(n_samples):
        response = call_llm(
            prompt=prompt + question,
            temperature=0.7
        )
        # 从回复中提取答案
        answer = extract_answer(response)
        answers.append(answer)

    # 多数投票
    from collections import Counter
    vote = Counter(answers)
    final_answer = vote.most_common(1)[0][0]
    confidence = vote.most_common(1)[0][1] / n_samples

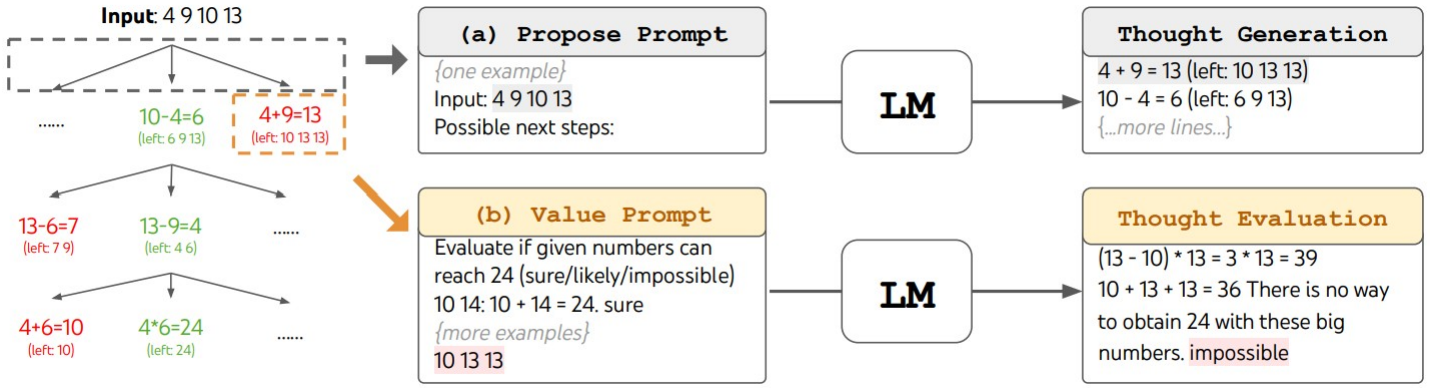
    return final_answer, confidence
```

X Wang et al.2022， Self-Consistency Improves Chain of Thought Reasoning in Language Models

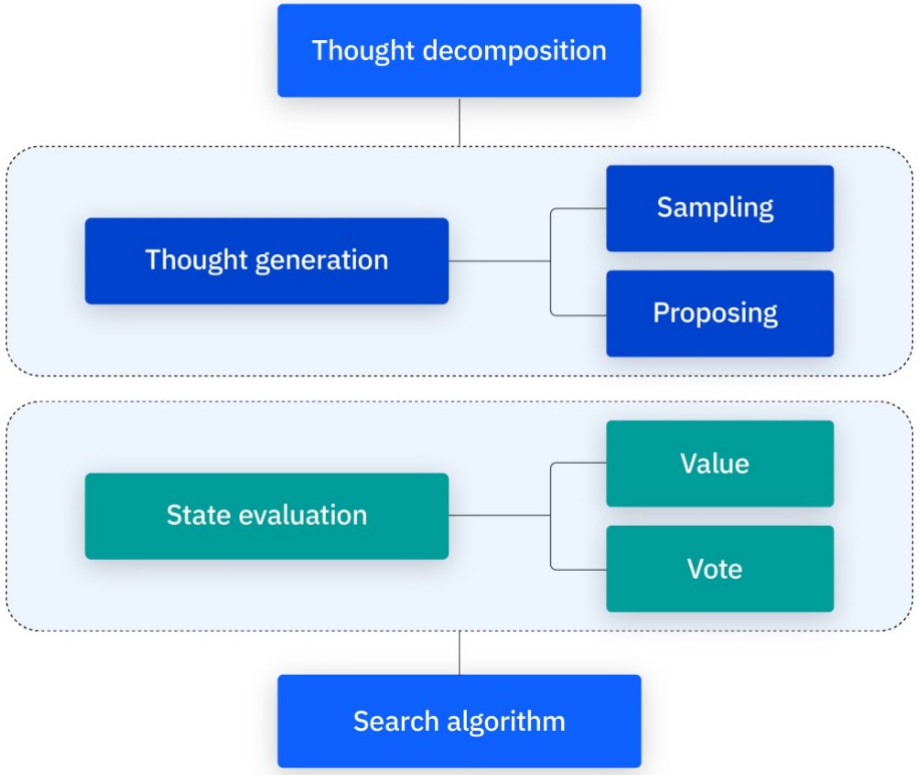
# ToT- 基本方法

思维链将推理过程表达为单一的线性链条，为解决更复杂的问题，思维树可以探索多个分支路径。

**示例**




**关键步骤**




S Yao et al.2023， Tree of Thoughts: Deliberate Problem Solving with Large Language Models

J Long 2023， Large Language Model Guided Tree-of-Thought

# ToT - 示例

## 提示词模板

```
Imagine three different experts answering this question.
All the experts write down the first step of their thought process and
share it with everyone.
Then, all the experts write down and share the next step of their thinking.
This continues until all the experts have written down every step of their
thought process.
If anyone noticesan error in an expert's step, that expert is removed from
the group.
The question is..
```

Hulbert et al. 2023

Using Tree-of-Thought Prompting to boost ChatGPT's reasoning

## PseudoCode

```
function ToT_DFS(task, x, node, depth, max_depth, args):
    // task: 任务对象
    // x: 问题输入
    // node: 当前节点
    // depth: 当前深度
    // max_depth: 最大推理深度
    // args: 配置参数

    if depth == max_depth:
        return node.value, node.text          // 到达最大深度，返回

    // ===== Phase 1: 思维生成 =====
    candidates = generate_candidates(task, x, node.text,
                                     n=args.n_generate_sample)

    // ===== Phase 2: 状态评估 =====
    for cand in candidates:
        cand.value = evaluate_state(task, x, cand.text,
                                    method=args.method_evaluate)

    // ===== Phase 3: 排序与剪枝 =====
    // 按评估值降序排序，优先探索高价值分支
    candidates.sort(key=lambda n: n.value, reverse=True)

    best_value = -inf
    best_solution = None

    for cand in candidates:
        if cand.value < args.prune_threshold:
            continue                          // 剪枝：放弃低价值分支[reference:20]

        // 递归探索该分支
        value, solution = ToT_DFS(task, x, cand, depth + 1,
                                  max_depth, args)

        if value > best_value:
            best_value = value
            best_solution = solution

    return best_value, best_solution

// 调用入口
function solve_DFS(task, args):
    x = task.get_input()
    root = Node(text='', depth=0, value=1.0)
    best_value, best_solution = ToT_DFS(task, x, root, 0,
                                        task.steps, args)
    return best_solution
```

# 多模态 Prompt

将文本、图片和视频等多模态输入内容作为 LLM/VLM 提示词的一部分

## 图像提示

- 图像预处理
  - 清晰度 150 DPI +
  - 分辨率 1024×768 +
  - 强化对比
  - 标注关键区域
- 典型场景
  - 图像识别与分类
  - 文本识别与提取
  - VQA
  - 艺术与美学分析

## 视频提示

- 采样频率
  - 短视频： 每秒 1 帧；
  - 中视频：每 5 秒一帧；
  - 长视频：每 30 秒一帧
- 多模态冲突消解
  - 视音频冲突：设置优先级，优先信任图像，其次信任文本
  - 多图像冲突：分析时间序列，识别变化原因和合理性
- 典型场景
  - 视频内容总结
  - 讲座 / 演讲笔记
  - 会议纪要
  - 字幕生成建议

# Prompt 调试优化

Prompt 测评

- 准确性、格式适配、一致性、鲁棒性、延迟

Prompt 调试方法

- 关键词 / 同义词替换
- temperature 优化

Prompt 调试工具

- PromptPerfect，提示词优化工具
- YPrompt ，提示词版本管理工具
- https://www.promptlayer.com/

针对特定模型版本的 Prompt 优化：

system_prompts_leaks



# Awesome Prompt

全球最大的开源 AI 提示词库 https://github.com/f/prompts.chat

Prompt 范例 https://github.com/ai-boost/awesome-prompts

Prompt 指南、论文和工具 https://github.com/promptslab/awesome-prompt-engineerin

research prompt https://github.com/Leey21/awesome-ai-research-writing

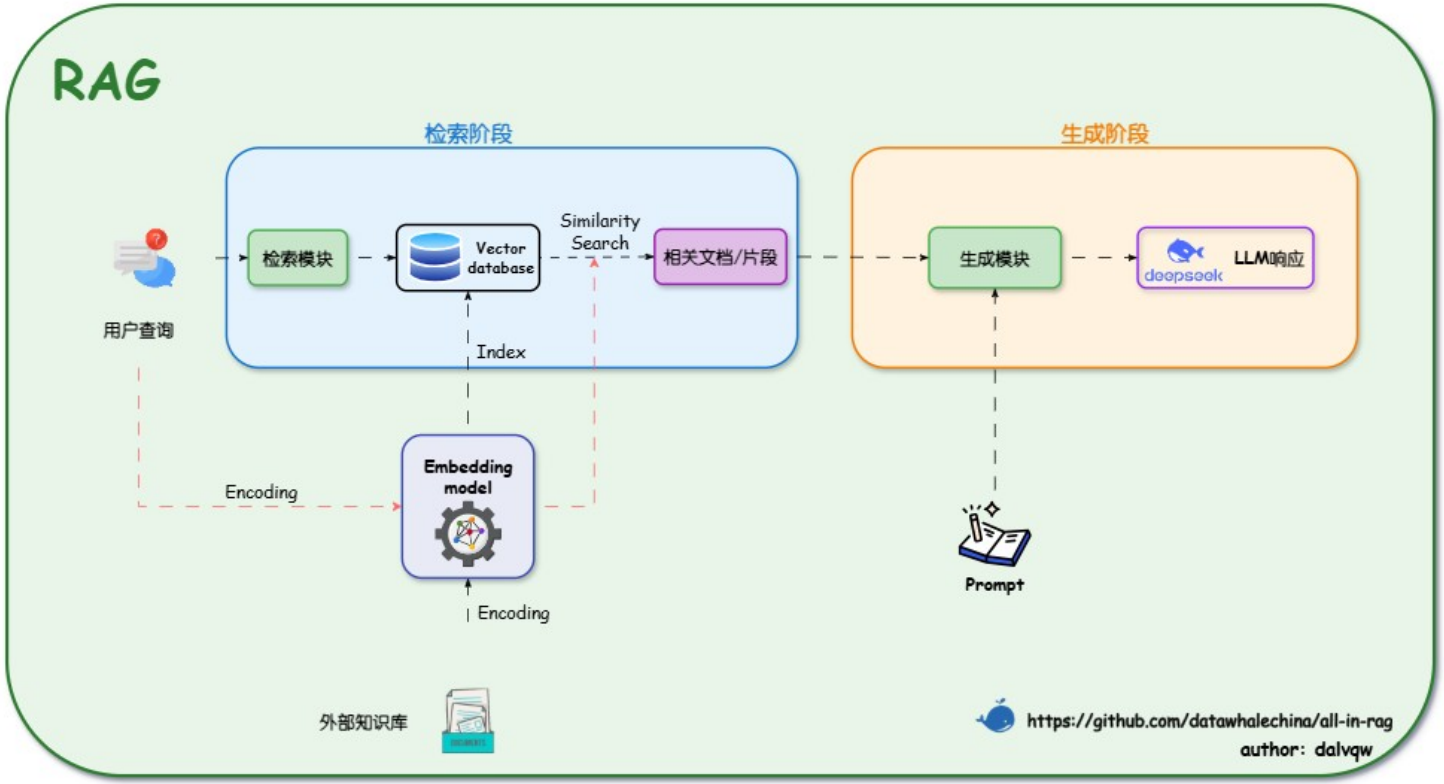


## Table of Contents


- 📋 Prompts — copy-paste ready
  - Coding & Development
  - DevOps & SRE
  - Data Engineering
  - AI & ML
  - Product & Strategy
  - Project Management
  - Healthcare & Clinical
  - Industrial & Automotive
  - Legal & Compliance
  - Knowledge & Documentation
  - Writing & Academic
  - Learning & Education
  - Research & Analysis
  - Productivity & Tasks
  - Safety & Compliance
  - Meta & Prompt Engineering
  - Image, Video & Audio Generation
  - Creative & Role-play
  - Game Development
  - Translation
  - Legacy (2023 era)
- 🔬 Frameworks — the engineering camp
  - Prompt Programming
  - Automatic Prompt Optimization
  - Tool Use & Reliability
  - Eval & Testing
  - Red Team & Security
  - Eval & Observability
  - Low-Code & Workflow Platforms
- 🕵️ System Prompt Leaks — learn from production
- 🧠 Prompt Engineering — techniques & defense
- 🔭 Context Engineering
- 🤖 Agent Ecosystem — MCP, Skills, Harness

# RAG - 基本方法

检索增强生成（Retrieval-Augmented Generation, RAG）是由 Lewis 等人在 2020 年提出的架构，已成为解决大语言模型“幻觉”和“知识过时”问题的标准解决方案。




**检索阶段**，寻找“非参数化知识”

- 知识向量化，对外部知识库进行编码
- 语义召回，通过相似度搜索从海量数据中精准锁定与问题最相关的文档片段。

**生成阶段**，融合两种知识

- 上下文整合，生成模块接收检索阶段送来的相关文档片段以及用户的原始问题。
- 指令引导生成，遵循预设的 Prompt 指令，进行可控的、有理有据的文本生成。

# RAG - 技术演进

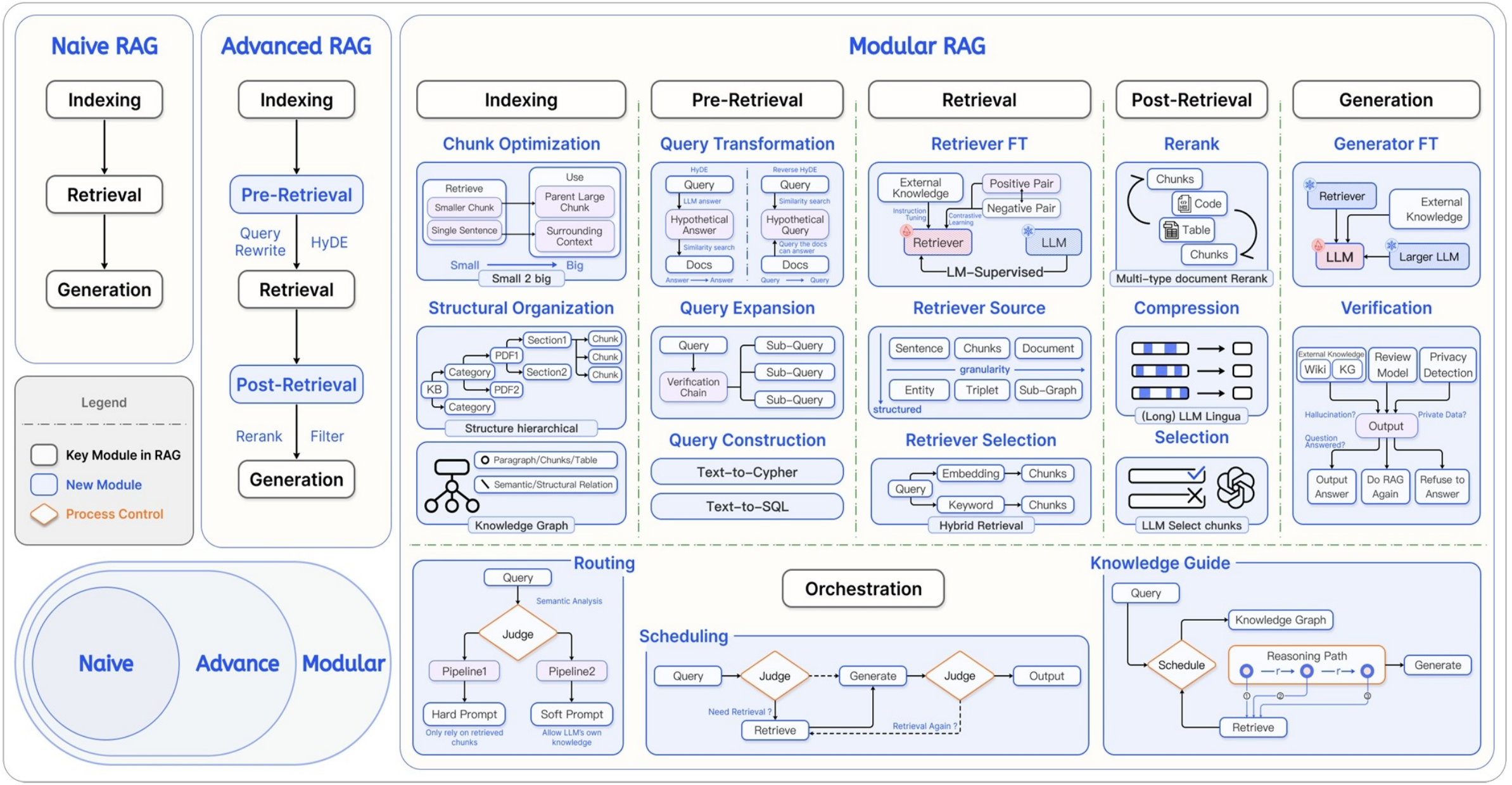



Lewis et al.2020， Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Asai et al.2023， Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection

Han et al.2025， Retrieval-Augmented Generation with Graphs (GraphRAG)

# RAG - 架构演进


Naive RAG
Indexing
Retrieval
Generation
Advanced RAG
Indexing
Pre-Retrieval
Query Rewrite
HyDE
Retrieval
Post-Retrieval
Rerank
Filter
Generation
Modular RAG
Indexing
Chunk Optimization
Retrieve
Smaller Chunk
Single Sentence
Use
Parent Large Chunk
Surrounding Context
Small
Big
Small 2 big
Structural Organization
KB
Category
PDF1
PDF2
Category
Section1
Section2
Chunk
Chunk
Chunk
Structure hierarchical
Paragraph/Chunks/Table
Semantic/Structural Relation
Knowledge Graph
Pre-Retrieval
Query Transformation
HyDE
Query
LLM answer
Hypothetical Answer
Similarity search
Docs
Answer
Answer
Reverse HyDE
Query
Similarity search
Hypothetical Query
Query the docs can answer
Docs
Query
Query
Query Expansion
Query
Verification Chain
Sub-Query
Sub-Query
Sub-Query
Query Construction
Text-to-Cypher
Text-to-SQL
Retrieval
Retriever FT
External Knowledge
Positive Pair
Negative Pair
Instruction Tuning
Contrastive Learning
Retriever
LLM
LM-Supervised
Retriever Source
Sentence
Chunks
Document
granularity
Entity
Triplet
Sub-Graph
structured
Retriever Selection
Query
Embedding
Chunks
Keyword
Chunks
Hybrid Retrieval
Post-Retrieval
Rerank
Chunks
Code
Table
Chunks
Multi-type document Rerank
Compression
(Long) LLM Lingua
Selection
LLM Select chunks
Generation
Generator FT
Retriever
External Knowledge
LLM
Larger LLM
Verification
External Knowledge
Wiki
KG
Review Model
Privacy Detection
Hallucination?
Output
Private Data?
Question Answered?
Output Answer
Do RAG Again
Refuse to Answer
Legend
Key Module in RAG
New Module
Process Control
Routing
Query
Semantic Analysis
Judge
Pipeline1
Pipeline2
Hard Prompt
Only rely on retrieved chunks
Soft Prompt
Allow LLM's own knowledge
Orchestration
Scheduling
Query
Judge
Generate
Judge
Output
Need Retrieval ?
Retrieve
Retrieval Again ?
Knowledge Guide
Query
Knowledge Graph
Schedule
Reasoning Path
Generate
Retrieve
Naive
Advance
Modular

# RAG - 检索阶段优化

**分块与索引优化**（Chunking 与 Indexing）

- 分块： Fixed-size； Content-aware； Structural

**查询转换**（Query Transformation）

- HyDE （ Hypothetical Document Embeddings）， LLM 先生成一个假设性回答，根据假设性回答去检索
- 多查询扩展（Multi-Query）， 将一个复杂问题拆解为多个子问题，并行执行搜索，然后去重合并结果

**混合检索** 关键词 + 向量

- 权重融合
- RRF 融合 $score(d) = \sum_{r \in R} \frac{1}{k + rank(d, r)}$

**重排序**（Reranking）

- 两阶段排序。初排 (Retrieval)： 从海量数据中得到 Top-K； 重排 (Reranking)： 对初排结果进行精细打分。
- Bi-Encoder 、 Cross-Encoder 、 ColBERT 、 LLM Reranking

**上下文组装**（Context Construction）

- Lost in the Middle 、 Long Context

# RAG - 生成阶段优化

## RAG 提示词模板

```
# Role
You are an expert support assistant for [Company Name].

# Task
Answer the user's question based strictly on the provided context.

# Context
<documents>
    <document index="1">
        <source>employee_handbook.pdf</source>
        <content>报销必须在费用发生后的 30 天内提交...</content>
    </document>
    <document index="2">
        <source>travel_policy.docx</source>
        <content>所有超过 5000 元的差旅费需要 VP 审批...</content>
    </document>
    ......
</documents>

# Rules

1. **Evidence-Based**: Every sentence in your answer must be supported by the context.
2. **Citation**: Cite sources using [index] format at the end of sentences.
3. **No Hallucination**: If the answer is not in the context, say "I cannot find the answer in the knowledge base."
4. **Tone**: Helpful, concise, and professional.

# User Question
{query_str}

# Answer
```

## 生成阶段优化方法

提示词

- 引用与溯源
- 明确禁止”脑补”
- 信息冲突解决策略

其他策略

- 引入外部知识校验
- 验证评估

# TongAgents-Prompt Template

*Prompt Template 是从静态文本到参数化指令*

**之前 · 静态 Prompt**

*"请帮我写一封营销邮件*
*介绍一家新开咖啡店*
*店铺名为「一杯花」。"*

模板化

**之后 · 模板**

*"请帮我写一封营销邮件*
*介绍一家*
**{{ 商品类型 }}** *店铺，*
*店名为「***{{ 店名 }}***」。*
语气：**{{ 语气 }}** 字数：**{{ 字数 }}** 字

→ *复用 · 参数化 · 组合*

*模板将结构与内容分离 —— 把一次性 prompt 转化为可复用的指令。*

# 五大主流范式

*每种范式对 prompt 模板的结构不同*

## 1 ReAct

*推理 + 行动*
*循环*

Yao 2022

思考：
推理
行动：
调用工具
观察：
读结果

## 2 Tool-Use

*LLM 选择*
*合适的工具*

OpenAI/Anthropic

工具表：
JSON schema
输出：
{tool,args}
框架执行

## 3 Reflection

*自我批评*
*与修订*

Shinn 2023

生成
自我批评
发现错误
修订
迭代

## 4 Plan-Execute

*先规划*
*再执行*

Wei 2022

分解任务
构建 DAG
执行
校验
输出

## 5 Multi-Agent

*角色分工*
*协同辩论*

Du 2023

多模型
独立角色
消息传递
共识
投票

*五大范式共享同一套 Prompt Template 基础 — 仅工作流结构不同。*

# TongAgents- Prompt Template

*ReAct 实战：模板如何引导 Agent 完成推理 + 行动循环*

## 1 ① 循环

1. **思考**
   推理下一步该做什么
2. **行动**
   调用工具
3. **观察**
   读取工具输出
4. **重复**
   直到任务完成

## 2 ② 模板

```
# ReAct 提示词模板

你是 {{ 智能体名称 }}。

# 可用工具
{{ 工具描述 }}

# 工作流
  思考：推理
  行动：调用工具
  观察：读取结果

# 输出格式
思考： < 推理过程 >
行动： { ... }

# 任务
{{ 用户问题 }}
```

## 3 ③ 实时执行

**思考 1**
需要天气 → 调用工具

**行动 1**
{" 工具 ":" 获取天气 "}

**观察 1**
{" 温度 ":22," 天气 ":" 晴 "}

**思考 2**
已得到数据 → 回答

**最终**
" 北京当前 22°C 晴☀ "

*模板强制结构化对话 —— 让每一步都可解释、可审计。*

# TongAgents - Prompt Template

*Tong-Agent 在生产环境中如何实现 Prompt Template*

## 1 ① 两种实现方式

### 内联模式 (string.Template)

```
from tongagents.prompt.template import PromptTemplate
# 创建模板
tmpl = PromptTemplate(
    '你是 $name。任务: $task'
)
# 渲染
prompt = tmpl.render(
    name='机器人',
    task='查询天气'
)
```

### 文件模式 (Jinja2 .j2)

```
from jinja2 import (
    Environment,
    FileSystemLoader
)
env = Environment(
    loader=FileSystemLoader(
        'tongagents/prompts/')
)
tmpl = env.get_template(
    'react_agent.j2')
```

## 2 ② 五步工作流




*Tong-Agents 使用 Jinja2 将模板逻辑与执行分离 —— 让 prompt 可维护、可测试。*

# 课后习题

**作业目标**

利用学习到的 prompt template 技术，实现一个大学教授人设，可以是严谨考究，也可以是亲和幽默的人设。

**核心任务**

学会利用 prompt template 来组织和实现 agent 的 system prompt

**关键要求**

✓必须用 template 机制来管理和实现 prompt

✓ demo 端到端跑通， prompt 生效

**最终效果**

调用 Agent ，输入 query： 请教您关于 llm 中 transform 相关的知识，可以根据不同的人设输出特定的风格回答

**学完你将掌握**

利用 prompt template 进行 prompt 管理的技巧